MindLink

Backend Implementation Guide — v2

INFME Assignment Edition

Node.js · Express · TypeScript · MongoDB · Mongoose · JWT · bcryptjs

Stack	Node.js + Express + TypeScript
Database	MongoDB + Mongoose
Auth	JWT + bcryptjs
Entities	User, Psicologo, Paciente, Consulta, Questionario, Desafio, Mensagem, Forum, MensagemForum, Alerta
Interop	XML export + XSD validation
Testing	Bruno (18-step flow)
Assignment	INFME — full CRUD, auth, permissions, XML/XSD

STEP 0 Project Setup — Folder Structure & Dependencies

Create the following folder structure inside your MindLink project. All paths below are relative to the project root.

```
backend-mindlink/
  auth/
    VerifyToken.ts          ← JWT verification middleware
    VerifyTokenByRole.ts    ← Role-based access middleware
  models/
    user.ts                 ← Base user (nome, email, password, tipo)
    psicologo.ts            ← Psychologist profile
    paciente.ts             ← Patient profile + formulario
    consulta.ts             ← Appointments
    questionario.ts         ← Daily mood/symptom entries
    desafio.ts              ← Challenges assigned by psicologo
    mensagem.ts             ← 1-to-1 chat between paciente & psicologo
    forum.ts                ← Forum thread/topic
    mensagemForum.ts        ← Post inside a forum thread
    alerta.ts               ← System alerts (auto-generated or manual)
  routes/
    UserRoutes.ts
    PsicologoRoutes.ts
    PacienteRoutes.ts
    ConsultaRoutes.ts
    QuestionarioRoutes.ts
    DesafioRoutes.ts
    MensagemRoutes.ts
    ForumRoutes.ts
    AlertaRoutes.ts
    AnaliseRoutes.ts
    ExportRoutes.ts
  utils/
    analiseHumor.ts         ← Rule-based mood analysis logic
    xmlExport.ts            ← XML generation helpers
    xmlValidation.ts        ← XSD validation helpers
  xml/
    schemas/
      historicoPaciente.xsd ← XSD schema definition
    exports/                ← Generated XML files saved here
  server.ts
  .env                      ← Secrets — NEVER commit this
  .gitignore
  package.json
  tsconfig.json
```

Install all dependencies

```
npm install express mongoose bcryptjs jsonwebtoken dotenv cors xmlbuilder2
npm install --save-dev @types/express @types/node @types/bcryptjs \
  @types/jsonwebtoken @types/cors ts-node typescript nodemon
```

For XML validation (XSD), install libxmljs2

```
npm install libxmljs2
npm install --save-dev @types/libxmljs2
```

.env file (create in root — add to .gitignore immediately)

```
PORT=5000
MONGO_URI=mongodb+srv://<user>:<pass>@mindlink.c6xgmy8.mongodb.net/?appName=mindLink
JWT_SECRET=<generate-a-long-random-string-here>
```

Nota: Generate a strong JWT secret: node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

.gitignore — minimum required entries

```
node_modules/
.env
xml/exports/
dist/
```

server.ts — register all routes

```
import express = require('express');
import mongoose = require('mongoose');
import bodyParser = require('body-parser');
import cors = require('cors');
import dotenv = require('dotenv');
dotenv.config();

const app = express();
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
app.use(cors());

mongoose.connect(process.env.MONGO_URI!)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error(err));

app.use('/api/users',        require('./routes/UserRoutes').userRoutes);
app.use('/api/psicologos',   require('./routes/PsicologoRoutes').psicologoRoutes);
app.use('/api/pacientes',    require('./routes/PacienteRoutes').pacienteRoutes);
app.use('/api/consultas',    require('./routes/ConsultaRoutes').consultaRoutes);
app.use('/api/questionarios',require('./routes/QuestionarioRoutes').questionarioRoutes);
app.use('/api/desafios',     require('./routes/DesafioRoutes').desafioRoutes);
app.use('/api/mensagens',    require('./routes/MensagemRoutes').mensagemRoutes);
app.use('/api/forum',        require('./routes/ForumRoutes').forumRoutes);
app.use('/api/alertas',      require('./routes/AlertaRoutes').alertaRoutes);
app.use('/api/analise',      require('./routes/AnaliseRoutes').analiseRoutes);
app.use('/api/export',       require('./routes/ExportRoutes').exportRoutes);

app.listen(process.env.PORT || 5000, () =>
  console.log(`Server running on port ${process.env.PORT || 5000}`));
```

STEP 1

Data Models — All 10 Mongoose Schemas

1.1 — User (models/user.ts)

Base account for every person in the system. The tipo field drives role-based access throughout the entire API.

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  nome:            { type: String,   required: true },
  genero:          { type: String,   required: true },
  data_nascimento: { type: Date,     required: true },
  email:           { type: String,   required: true, unique: true,
                     match: [/^\S+@\S+\.\S+$/, 'Email inválido'] },
  password:        { type: String,   required: true },
  tipo:            { type: String,   required: true,
                     enum: ['psicologo', 'paciente', 'admin'] },
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

1.2 — Psicologo (models/psicologo.ts)

```
const psicologoSchema = new mongoose.Schema({
  user:         { type: mongoose.Schema.Types.ObjectId,
                  ref: 'User', required: true },
  especialidade:{ type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model('Psicologo', psicologoSchema);
```

1.3 — Paciente (models/paciente.ts)

```
const pacienteSchema = new mongoose.Schema({
  user:      { type: mongoose.Schema.Types.ObjectId, ref: 'User',      required: true },
  psicologo: { type: mongoose.Schema.Types.ObjectId, ref: 'Psicologo', required: true },
  doenca:    { type: String, required: true },
  formulario: {
    historicoMedico: {
      comorbilidades: [{ type: String }],
    },
    estiloDeVida: {
      exercicioRegular: { type: Boolean, default: null },
      fumador:          { type: Boolean, default: null },
    },
    historicoFamiliar: {
      cancro:        { type: Boolean, default: null },
      doencaCardiaca:{ type: Boolean, default: null },
    },
  },
}, { timestamps: true });

module.exports = mongoose.model('Paciente', pacienteSchema);
```

1.4 — Questionario (models/questionario.ts)

Daily mood and symptom entry submitted by the paciente. humor is 1–5 (1 = very low, 5 = excellent). This model feeds the automatic analysis module.

```
const questionarioSchema = new mongoose.Schema({
  paciente: { type: mongoose.Schema.Types.ObjectId,
              ref: 'Paciente', required: true },
  data:     { type: Date, required: true, default: Date.now },
  humor:    { type: Number, required: true, min: 1, max: 5 },
  sintomas: { type: String },
  notas:    { type: String },
}, { timestamps: true });

module.exports = mongoose.model('Questionario', questionarioSchema);
```

1.5 — Consulta (models/consulta.ts)

```
const consultaSchema = new mongoose.Schema({
  paciente:  { type: mongoose.Schema.Types.ObjectId,
               ref: 'Paciente',  required: true },
  psicologo: { type: mongoose.Schema.Types.ObjectId,
               ref: 'Psicologo', required: true },
  data:      { type: Date,   required: true },
  estado:    { type: String, required: true,
               enum: ['agendada', 'realizada', 'cancelada'],
               default: 'agendada' },
}, { timestamps: true });

module.exports = mongoose.model('Consulta', consultaSchema);
```

1.6 — Desafio (models/desafio.ts)

Challenges or therapeutic tasks assigned by a psicologo to a paciente. The sugestao field holds an optional recommendation message. This replaces a separate Sugestao entity — cleaner for an academic project.

```
const desafioSchema = new mongoose.Schema({
  paciente:    { type: mongoose.Schema.Types.ObjectId,
                 ref: 'Paciente',  required: true },
  psicologo:   { type: mongoose.Schema.Types.ObjectId,
                 ref: 'Psicologo', required: true },
  titulo:      { type: String, required: true },
  descricao:   { type: String, required: true },
  tipo:        { type: String, required: true, enum: ['diario', 'semanal'] },
  data_criacao:{ type: Date,   default: Date.now },
  data_inicio: { type: Date,   required: true },
  data_fim:    { type: Date,   required: true },
  estado:      { type: String, required: true,
                 enum: ['pendente', 'concluido'], default: 'pendente' },
  sugestao:    { type: String },
}, { timestamps: true });

module.exports = mongoose.model('Desafio', desafioSchema);
```

1.7 — Mensagem (models/mensagem.ts)

Private 1-to-1 messages between a paciente and their psicologo. The remetente field identifies who sent each message.

```
const mensagemSchema = new mongoose.Schema({
  paciente:   { type: mongoose.Schema.Types.ObjectId,
                ref: 'Paciente',  required: true },
  psicologo:  { type: mongoose.Schema.Types.ObjectId,
                ref: 'Psicologo', required: true },
  remetente:  { type: String, required: true,
                enum: ['paciente', 'psicologo'] },
  mensagem:   { type: String, required: true },
  data:       { type: Date,   default: Date.now },
}, { timestamps: true });

module.exports = mongoose.model('Mensagem', mensagemSchema);
```

1.8 — Forum (models/forum.ts)

A forum thread/topic. Accessible by all authenticated pacientes. The autor field links to a User (paciente).

```
const forumSchema = new mongoose.Schema({
  titulo:    { type: String, required: true },
  descricao: { type: String },
  autor:     { type: mongoose.Schema.Types.ObjectId,
               ref: 'User', required: true },
  tags:      [{ type: String }],
}, { timestamps: true });

module.exports = mongoose.model('Forum', forumSchema);
```

1.9 — MensagemForum (models/mensagemForum.ts)

An individual post inside a Forum thread.

```
const mensagemForumSchema = new mongoose.Schema({
  forum:    { type: mongoose.Schema.Types.ObjectId,
              ref: 'Forum', required: true },
  autor:    { type: mongoose.Schema.Types.ObjectId,
              ref: 'User',  required: true },
  conteudo: { type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model('MensagemForum', mensagemForumSchema);
```

1.10 — Alerta (models/alerta.ts)

System alerts created automatically by the analysis module or manually by a psicologo. The lido flag tracks whether the alert has been acknowledged.

```
const alertaSchema = new mongoose.Schema({
  paciente:  { type: mongoose.Schema.Types.ObjectId,
               ref: 'Paciente',  required: true },
  psicologo: { type: mongoose.Schema.Types.ObjectId,
               ref: 'Psicologo' },
  tipo:      { type: String, required: true,
               enum: ['humor_baixo', 'urgente', 'consulta', 'sintomas'] },
  mensagem:  { type: String, required: true },
  nivel:     { type: String, required: true,
               enum: ['baixo', 'medio', 'alto'] },
  lido:      { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model('Alerta', alertaSchema);
```

STEP 2

Authentication Middleware

2.1 — auth/VerifyToken.ts

Add this middleware to any route that requires the user to be logged in. It reads the Bearer token from the Authorization header, verifies it, and attaches the decoded payload (id + tipo) to req.user.

```
const jwt = require('jsonwebtoken');

const verifyToken = (req: any, res: any, next: any) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // "Bearer <token>"

  if (!token)
    return res.status(401).json({ error: 'Token não fornecido' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!);
    req.user = decoded; // { id, tipo }
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token inválido ou expirado' });
  }
};

module.exports = { verifyToken };
```

2.2 — auth/VerifyTokenByRole.ts

Use this middleware after verifyToken to restrict a route to one or more specific tipos. Pass one or more allowed roles as arguments.

```
const verifyTokenByRole = (...roles: string[]) => {
  return (req: any, res: any, next: any) => {
    if (!req.user)
      return res.status(401).json({ error: 'Não autenticado' });

    if (!roles.includes(req.user.tipo))
      return res.status(403).json({
        error: 'Acesso negado: não tem permissão para este recurso'
      });

    next();
  };
};

module.exports = { verifyTokenByRole };
```

How to use both middlewares together on a route:

```
const { verifyToken }        = require('../auth/VerifyToken');
const { verifyTokenByRole }  = require('../auth/VerifyTokenByRole');

// Only psicologos can access this route:
router.post('/', verifyToken, verifyTokenByRole('psicologo'), createConsulta);

// Both types can access this route:
router.get('/:id', verifyToken, verifyTokenByRole('psicologo','paciente'), getById);
```

STEP 3

Security — bcrypt + JWT in UserRoutes.ts

3.1 — Register: hash password before saving

```
const bcrypt  = require('bcryptjs');
const jwt     = require('jsonwebtoken');
const User    = require('../models/user');

router.post('/register', async (req: any, res: any) => {
  try {
    const { nome, genero, data_nascimento, email, password, tipo } = req.body;

    // 1. Check for duplicate email
    const exists = await User.findOne({ email });
    if (exists)
      return res.status(400).json({ error: 'Email já registado' });

    // 2. Hash the password (NEVER store plain text)
    const hashed = await bcrypt.hash(password, 12);

    // 3. Create and save user
    const user = new User({ nome, genero, data_nascimento, email,
                            password: hashed, tipo });
    await user.save();

    const safeUser = user.toObject();
    delete safeUser.password;
    res.status(201).json(safeUser);

  } catch (err) {
    res.status(400).json({ error: (err as Error).message });
  }
});
```

3.2 — Login: compare hash + return JWT

```
router.post('/login', async (req: any, res: any) => {
  try {
    const { email, password } = req.body;

    // 1. Find user by email
    const user = await User.findOne({ email });
    if (!user)
      return res.status(401).json({ error: 'Email ou password incorretos' });

    // 2. Compare password with stored hash
    const match = await bcrypt.compare(password, user.password);
    if (!match)
      return res.status(401).json({ error: 'Email ou password incorretos' });

    // 3. Sign JWT — include id and tipo (required for role middleware)
    //    JWT_SECRET must come from .env — NEVER hardcode a fallback in production
    const token = jwt.sign(
      { id: user._id, tipo: user.tipo },
      process.env.JWT_SECRET!,
      { expiresIn: '7d' }
    );

    const safeUser = user.toObject();
    delete safeUser.password;
    res.json({ token, user: safeUser });

  } catch (err) {
    res.status(500).json({ error: (err as Error).message });
  }
});
```

STEP 4

Complete CRUD Routes — All Entities

Each route file follows the same pattern. Below is the template for PsicologoRoutes.ts as a reference — all other files follow the same structure. After the template, a full table lists every route across all entities.

Route file template — PsicologoRoutes.ts

```
const express  = require('express');
const router   = express.Router();
const Psicologo = require('../models/psicologo');
const { verifyToken }       = require('../auth/VerifyToken');
const { verifyTokenByRole } = require('../auth/VerifyTokenByRole');

// POST /api/psicologos — create psicologo profile (admin or self)
router.post('/', verifyToken, async (req: any, res: any) => {
  try {
    const { user, especialidade } = req.body;
    const p = new Psicologo({ user, especialidade });
    await p.save();
    res.status(201).json(p);
  } catch (err) { res.status(400).json({ error: (err as Error).message }); }
});

// GET /api/psicologos — list all (protected)
router.get('/', verifyToken, async (req: any, res: any) => {
  try {
    const list = await Psicologo.find().populate('user', '-password');
    res.json(list);
  } catch (err) { res.status(500).json({ error: (err as Error).message }); }
});

// GET /api/psicologos/:id
router.get('/:id', verifyToken, async (req: any, res: any) => {
  try {
    const p = await Psicologo.findById(req.params.id).populate('user','-password');
    if (!p) return res.status(404).json({ error: 'Psicologo não encontrado' });
    res.json(p);
  } catch (err) { res.status(500).json({ error: (err as Error).message }); }
});

// PUT /api/psicologos/:id — update (psicologo only, own profile)
router.put('/:id', verifyToken, verifyTokenByRole('psicologo','admin'),
  async (req: any, res: any) => {
    try {
      const p = await Psicologo.findByIdAndUpdate(req.params.id, req.body,
                  { new: true, runValidators: true });
      if (!p) return res.status(404).json({ error: 'Psicologo não encontrado' });
      res.json(p);
    } catch (err) { res.status(400).json({ error: (err as Error).message }); }
  }
);

// DELETE /api/psicologos/:id
router.delete('/:id', verifyToken, verifyTokenByRole('admin'),
  async (req: any, res: any) => {
    try {
      await Psicologo.findByIdAndDelete(req.params.id);
      res.json({ message: 'Psicologo removido' });
    } catch (err) { res.status(500).json({ error: (err as Error).message }); }
  }
);

module.exports = { psicologoRoutes: router };
```

Full route map — all entities

Entity	Method	Endpoint	Auth	Permitted Roles
User	POST	/api/users/register	No	Public
User	POST	/api/users/login	No	Public
User	GET	/api/users	Yes	admin
User	GET	/api/users/:id	Yes	any
User	PUT	/api/users/:id	Yes	any (own)
User	DELETE	/api/users/:id	Yes	admin
Psicologo	POST	/api/psicologos	Yes	admin
Psicologo	GET	/api/psicologos	Yes	any
Psicologo	GET	/api/psicologos/:id	Yes	any
Psicologo	PUT	/api/psicologos/:id	Yes	psicologo, admin
Psicologo	DELETE	/api/psicologos/:id	Yes	admin
Paciente	POST	/api/pacientes	Yes	admin, psicologo
Paciente	GET	/api/pacientes	Yes	admin, psicologo
Paciente	GET	/api/pacientes/:id	Yes	psicologo (own), paciente (self)
Paciente	GET	/api/pacientes/psicologo/:id	Yes	psicologo (own)
Paciente	PUT	/api/pacientes/:id	Yes	psicologo (own), admin
Paciente	DELETE	/api/pacientes/:id	Yes	admin
Consulta	POST	/api/consultas	Yes	psicologo
Consulta	GET	/api/consultas	Yes	admin
Consulta	GET	/api/consultas/:id	Yes	psicologo, paciente
Consulta	GET	/api/consultas/paciente/:id	Yes	psicologo, paciente
Consulta	GET	/api/consultas/psicologo/:id	Yes	psicologo
Consulta	PUT	/api/consultas/:id	Yes	psicologo
Consulta	DELETE	/api/consultas/:id	Yes	psicologo, admin
Questionario	POST	/api/questionarios	Yes	paciente
Questionario	GET	/api/questionarios	Yes	admin
Questionario	GET	/api/questionarios/:id	Yes	psicologo, paciente
Questionario	GET	/api/questionarios/paciente/:id	Yes	psicologo, paciente (self)
Questionario	PUT	/api/questionarios/:id	Yes	paciente (own)
Questionario	DELETE	/api/questionarios/:id	Yes	admin
Desafio	POST	/api/desafios	Yes	psicologo
Desafio	GET	/api/desafios	Yes	admin
Desafio	GET	/api/desafios/:id	Yes	psicologo, paciente
Desafio	GET	/api/desafios/paciente/:id	Yes	psicologo, paciente (self)
Desafio	GET	/api/desafios/psicologo/:id	Yes	psicologo (own)
Desafio	PUT	/api/desafios/:id	Yes	psicologo
Desafio	PATCH	/api/desafios/:id/estado	Yes	psicologo, paciente

Desafio	**DELETE**	/api/desafios/:id	Yes	psicologo, admin
Mensagem	**POST**	/api/mensagens	Yes	psicologo, paciente
Mensagem	**GET**	/api/mensagens/conversa/:pacId/:psicoId	Yes	psicologo (own), paciente (self)
Mensagem	**GET**	/api/mensagens/:id	Yes	psicologo, paciente (parties only)
Mensagem	**PUT**	/api/mensagens/:id	Yes	sender only
Mensagem	**DELETE**	/api/mensagens/:id	Yes	sender or admin
Forum	**POST**	/api/forum	Yes	paciente
Forum	**GET**	/api/forum	Yes	paciente, psicologo
Forum	**GET**	/api/forum/:id	Yes	paciente, psicologo
Forum	**PUT**	/api/forum/:id	Yes	author only
Forum	**DELETE**	/api/forum/:id	Yes	author, admin
MensagemForum	**POST**	/api/forum/:forumId/mensagens	Yes	paciente, psicologo
MensagemForum	**GET**	/api/forum/:forumId/mensagens	Yes	paciente, psicologo
MensagemForum	**PUT**	/api/forum/mensagens/:id	Yes	author only
MensagemForum	**DELETE**	/api/forum/mensagens/:id	Yes	author, admin
Alerta	**POST**	/api/alertas	Yes	psicologo, system
Alerta	**GET**	/api/alertas	Yes	admin
Alerta	**GET**	/api/alertas/paciente/:id	Yes	psicologo (own), paciente (self)
Alerta	**GET**	/api/alertas/psicologo/:id	Yes	psicologo (own)
Alerta	**PATCH**	/api/alertas/:id/lido	Yes	psicologo, paciente
Alerta	**DELETE**	/api/alertas/:id	Yes	admin

STEP 5

Automatic Mood Analysis Module

The analysis module runs rule-based checks on a patient's recent Questionario records and automatically creates Alerta documents when risk patterns are detected. It lives in utils/analiseHumor.ts and is called from AnaliseRoutes.ts.

utils/analiseHumor.ts

```
const Questionario = require('../models/questionario');
const Alerta       = require('../models/alerta');

interface AlertaPayload {
  tipo: string; nivel: string; mensagem: string;
}

const analisarHumor = async (pacienteId: string): Promise<AlertaPayload[]> => {
  // Fetch the last 7 questionnaires, most recent first
  const registos = await Questionario
    .find({ paciente: pacienteId })
    .sort({ data: -1 })
    .limit(7);

  const alertas: AlertaPayload[] = [];

  // Rule 1 — humor == 1 in any entry → urgent alert
  const urgente = registos.find((q: any) => q.humor === 1);
  if (urgente) {
    alertas.push({
      tipo: 'urgente', nivel: 'alto',
      mensagem: 'Humor crítico (valor 1) registado. Contacto urgente recomendado.',
    });
  }

  // Rule 2 — humor <= 2 for 3 consecutive days → risk alert
  const ultimos3 = registos.slice(0, 3);
  if (ultimos3.length === 3 && ultimos3.every((q: any) => q.humor <= 2)) {
    alertas.push({
      tipo: 'humor_baixo', nivel: 'medio',
      mensagem: '3 dias consecutivos com humor baixo. Consulta recomendada.',
    });
  }

  // Rule 3 — repeated anxiety/stress/insomnia keywords → sintomas alert
  const keywords = ['ansiedade', 'stress', 'insónia', 'insonia', 'pânico'];
  const sintomas = registos
    .map((q: any) => (q.sintomas || '').toLowerCase())
    .join(' ');
  const matchCount = keywords.filter(k => sintomas.includes(k)).length;
  if (matchCount >= 2) {
    alertas.push({
      tipo: 'sintomas', nivel: 'baixo',
      mensagem: `Sintomas recorrentes detectados: ${keywords.filter(k =>
        sintomas.includes(k)).join(', ')}.`,
    });
  }

  // Persist alerts to the database
  for (const a of alertas) {
    await Alerta.create({ paciente: pacienteId, ...a });
  }

  return alertas;
};

module.exports = { analisarHumor };
```

routes/AnaliseRoutes.ts

```
const router        = require('express').Router();
const { analisarHumor }    = require('../utils/analiseHumor');
const { verifyToken }      = require('../auth/VerifyToken');
const { verifyTokenByRole }= require('../auth/VerifyTokenByRole');

// POST /api/analise/paciente/:pacienteId/run
// Trigger analysis manually (psicologo or system)
router.post('/paciente/:pacienteId/run',
  verifyToken,
  verifyTokenByRole('psicologo', 'admin'),
  async (req: any, res: any) => {
    try {
      const alertas = await analisarHumor(req.params.pacienteId);
      res.json({ alertasGerados: alertas.length, alertas });
    } catch (err) {
      res.status(500).json({ error: (err as Error).message });
    }
  }
);

module.exports = { analiseRoutes: router };
```

STEP 6

XML Export & XSD Validation (INFME Requirement)

This step satisfies the interoperability requirement of the INFME assignment. The system can export a patient's full history as a validated XML document.

Install xmlbuilder2 and libxmljs2 (if not done in Step 0)

```
npm install xmlbuilder2 libxmljs2
npm install --save-dev @types/libxmljs2
```

6.1 — xml/schemas/historicoPaciente.xsd

Create this file at xml/schemas/historicoPaciente.xsd. It defines the structure that all exported XML must conform to. Part 1: root element and container types.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Root element -->
  <xs:element name="historicoPaciente">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="paciente"      type="PacienteType"/>
        <xs:element name="questionarios" type="QuestionariosType"/>
        <xs:element name="consultas"     type="ConsultasType" minOccurs="0"/>
        <xs:element name="alertas"       type="AlertasType"   minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="exportDate" type="xs:dateTime" use="required"/>
    </xs:complexType>
  </xs:element>

  <!-- Paciente -->
  <xs:complexType name="PacienteType">
    <xs:sequence>
      <xs:element name="id"     type="xs:string"/>
      <xs:element name="nome"   type="xs:string"/>
      <xs:element name="doenca" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Questionarios container -->
  <xs:complexType name="QuestionariosType">
    <xs:sequence>
      <xs:element name="questionario" type="QuestionarioType"
                  minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Single questionario entry -->
  <xs:complexType name="QuestionarioType">
    <xs:sequence>
      <xs:element name="data"     type="xs:dateTime"/>
      <xs:element name="humor"    type="xs:integer"/>
      <xs:element name="sintomas" type="xs:string" minOccurs="0"/>
      <xs:element name="notas"    type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
```

Part 2: consultas and alertas type definitions (same file, continues below).

```
  <!-- Consultas container -->
  <xs:complexType name="ConsultasType">
    <xs:sequence>
      <xs:element name="consulta" type="ConsultaType"
                  minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Single consulta -->
  <xs:complexType name="ConsultaType">
    <xs:sequence>
      <xs:element name="data"   type="xs:dateTime"/>
      <xs:element name="estado" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Alertas container -->
  <xs:complexType name="AlertasType">
    <xs:sequence>
      <xs:element name="alerta" type="AlertaType"
                  minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Single alerta -->
  <xs:complexType name="AlertaType">
    <xs:sequence>
      <xs:element name="tipo"     type="xs:string"/>
      <xs:element name="nivel"    type="xs:string"/>
      <xs:element name="mensagem" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

</xs:schema>
```

6.2 — utils/xmlExport.ts

```
const { create } = require('xmlbuilder2');
const fs          = require('fs');
const path        = require('path');

const exportHistoricoPaciente = (
  paciente: any, questionarios: any[],
  consultas: any[] = [], alertas: any[] = []
): string => {
  const root = create({ version: '1.0', encoding: 'UTF-8' })
    .ele('historicoPaciente', { exportDate: new Date().toISOString() });

  root.ele('paciente')
    .ele('id').txt(paciente._id.toString()).up()
    .ele('nome').txt(paciente.user?.nome || '').up()
    .ele('doenca').txt(paciente.doenca).up()
    .up();

  const qEle = root.ele('questionarios');
  for (const q of questionarios) {
    qEle.ele('questionario')
      .ele('data').txt(new Date(q.data).toISOString()).up()
      .ele('humor').txt(String(q.humor)).up()
      .ele('sintomas').txt(q.sintomas || '').up()
      .ele('notas').txt(q.notas || '').up()
      .up();
  }

  const cEle = root.ele('consultas');
  for (const c of consultas) {
    cEle.ele('consulta')
      .ele('data').txt(new Date(c.data).toISOString()).up()
      .ele('estado').txt(c.estado).up()
      .up();
  }

  const aEle = root.ele('alertas');
  for (const a of alertas) {
    aEle.ele('alerta')
      .ele('tipo').txt(a.tipo).up()
      .ele('nivel').txt(a.nivel).up()
      .ele('mensagem').txt(a.mensagem).up()
      .up();
  }

  const xml = root.end({ prettyPrint: true });

  // Save a copy to xml/exports/
  const filename = `paciente_${paciente._id}_${Date.now()}.xml`;
  const exportPath = path.join(__dirname, '../xml/exports', filename);
  fs.writeFileSync(exportPath, xml, 'utf8');

  return xml;
};

module.exports = { exportHistoricoPaciente };
```

6.3 — utils/xmlValidation.ts

```
const libxml = require('libxmljs2');
const fs     = require('fs');
const path   = require('path');

const validateXml = (xmlString: string): { valid: boolean; errors: string[] } => {
  const xsdPath = path.join(__dirname, '../xml/schemas/historicoPaciente.xsd');
  const xsdString = fs.readFileSync(xsdPath, 'utf8');

  const xmlDoc = libxml.parseXml(xmlString);
  const xsdDoc = libxml.parseXml(xsdString);

  const valid = xmlDoc.validate(xsdDoc);
  const errors = xmlDoc.validationErrors.map((e: any) => e.message);

  return { valid, errors };
};

module.exports = { validateXml };
```

6.4 — routes/ExportRoutes.ts

```
const router    = require('express').Router();
const Paciente  = require('../models/paciente');
const Questionario = require('../models/questionario');
const Consulta  = require('../models/consulta');
const Alerta    = require('../models/alerta');
const { exportHistoricoPaciente } = require('../utils/xmlExport');
const { validateXml }            = require('../utils/xmlValidation');
const { verifyToken }            = require('../auth/VerifyToken');
const { verifyTokenByRole }      = require('../auth/VerifyTokenByRole');

// GET /api/export/paciente/:id/historico/xml
router.get('/paciente/:id/historico/xml',
  verifyToken,
  verifyTokenByRole('psicologo', 'admin'),
  async (req: any, res: any) => {
    try {
      const paciente      = await Paciente.findById(req.params.id)
                              .populate('user', '-password');
      const questionarios = await Questionario.find({ paciente: req.params.id });
      const consultas     = await Consulta.find({ paciente: req.params.id });
      const alertas       = await Alerta.find({ paciente: req.params.id });

      const xml = exportHistoricoPaciente(paciente, questionarios,
                    consultas, alertas);

      // Validate before sending
      const { valid, errors } = validateXml(xml);
      if (!valid)
        return res.status(500).json({ error: 'XML inválido', errors });

      res.set('Content-Type', 'application/xml');
      res.send(xml);
    } catch (err) {
      res.status(500).json({ error: (err as Error).message });
    }
  }
);

module.exports = { exportRoutes: router };
```

STEP 7

Data Validation & Error Handling

Apply these practices consistently across all route files.

Mongoose-level validation (enforced automatically)

- required: true — field must be present
- unique: true — no duplicate values (e.g., email)
- enum: [...] — value must be one of the listed options
- min / max — numeric range (e.g., humor: min 1, max 5)
- match: /regex/ — format check (e.g., email pattern)
- runValidators: true — enforce schema rules on update operations

Route-level validation example (checking ObjectId before querying)

```
const mongoose = require('mongoose');

// At the top of any route that uses :id
if (!mongoose.Types.ObjectId.isValid(req.params.id))
  return res.status(400).json({ error: 'ID inválido' });
```

Standard error response format — use this everywhere

```
// 400 — bad request / validation error
res.status(400).json({ error: 'Campo obrigatório em falta: nome' });

// 401 — not authenticated
res.status(401).json({ error: 'Token não fornecido ou inválido' });

// 403 — authenticated but not authorised
res.status(403).json({ error: 'Acesso negado: sem permissão para este recurso' });

// 404 — resource not found
res.status(404).json({ error: 'Paciente não encontrado' });

// 409 — conflict (e.g. duplicate email)
res.status(409).json({ error: 'Email já registado' });

// 500 — server error
res.status(500).json({ error: 'Erro interno do servidor' });
```

Global error handler in server.ts (add after all routes)

```
app.use((err: any, req: any, res: any, next: any) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Erro interno do servidor' });
});
```

STEP 8 Bruno Testing Flow — 18 Steps

Create a collection called MindLink API in Bruno. Add one folder per entity. Execute requests in the order below — each step depends on data created in the previous one. Save IDs returned in each response to use in later requests.

1	**Register psicologo User** POST /api/users/register	`{ "nome":"Dr. Ana", "genero":"F", "data_nascimento":"1985-03-10", "email":"ana@clinic.pt", "password":"pass123", "tipo":"psicologo" }`	Save returned _id as USER_PSI_ID
2	**Create Psicologo profile** POST /api/psicologos	`{ "user": "", "especialidade": "Psicologia Clínica" }`	Save returned _id as PSI_ID
3	**Register paciente User** POST /api/users/register	`{ "nome":"João Silva", "genero":"M", "data_nascimento":"1995-06-20", "email":"joao@email.pt", "password":"pass456", "tipo":"paciente" }`	Save returned _id as USER_PAC_ID
4	**Create Paciente profile** POST /api/pacientes	`{ "user":"", "psicologo":"", "doenca":"Ansiedade" }`	Save returned _id as PAC_ID
5	**Login as psicologo** POST /api/users/login	`{ "email":"ana@clinic.pt", "password":"pass123" }`	Save token as TOKEN_PSI — add to header: Authorization: Bearer
6	**Test protected route without token** GET /api/pacientes	`No Authorization header`	Expect 401 Unauthorized
7	**Test protected route with token** GET /api/pacientes	`Authorization: Bearer`	Expect 200 with list of pacientes
8	**Login as paciente, submit Questionario** POST /api/questionarios	`{ "paciente":"", "humor":2, "sintomas":"ansiedade", "notas":"dia difícil" }`	Submit 3 times with humor <= 2 to trigger analysis rules
9	**Run automatic analysis** POST /api/analise/paciente//run	`Authorization: Bearer`	Expect JSON with generated alerts
10	**Confirm Alerta was created** GET /api/alertas/paciente/	`Authorization: Bearer`	Expect at least one alerta with nivel: "medio" or "alto"
11	**Create Consulta** POST /api/consultas	`{ "paciente":"", "psicologo":"", "data":"2026-06-01", "estado":"agendada" }`	Save returned _id as CONSULTA_ID

12	**Create Desafio** POST /api/desafios	`{ "paciente":"", "psicologo":"", "titulo":"Diário de Gratidão", "descricao":"Escreve 3 coisas boas por dia", "tipo":"diario", "data_inicio":"2026-05-20", "data_fim":"2026-05-27" }`	Save returned _id as DESAFIO_ID
13	**Mark Desafio as completed** PATCH /api/desafios//estado	`{ "estado": "concluido" }`	Expect updated desafio with estado: concluido
14	**Send a Mensagem (psicologo to paciente)** POST /api/mensagens	`{ "paciente":"", "psicologo":"", "remetente":"psicologo", "mensagem":"Como correu a semana?" }`	Expect 201 Created
15	**Create a Forum topic** POST /api/forum	`{ "titulo":"Dicas para gerir ansiedade", "descricao":"Partilha as tuas estratégias", "autor":"" }`	Save returned _id as FORUM_ID
16	**Post a MensagemForum reply** POST /api/forum//mensagens	`{ "autor":"", "conteudo":"Respiração profunda ajuda muito!" }`	Expect 201 Created
17	**Export XML history** GET /api/export/paciente//historico/xml	`Authorization: Bearer`	Expect application/xml response. Save to file historico.xml
18	**Validate XML against XSD** GET /api/export/paciente//historico/xml	`The server validates automatically — check response for errors field`	Expect valid:true in server logs; if invalid, check utils/xmlValidation.ts

STEP 9

Final Checklist — Backend Complete

Setup & Structure

- ■ Folder structure created as per Step 0
- ■ All dependencies installed (express, mongoose, bcryptjs, jwt, dotenv, cors, xmlbuilder2)
- ■ .env contains MONGO_URI and JWT_SECRET (no hardcoded fallback)
- ■ .env is in .gitignore
- ■ server.ts registers all 11 route groups

Models

- ■ User — nome, genero, data_nascimento, email, password, tipo
- ■ Psicologo — user (ref), especialidade
- ■ Paciente — user (ref), psicologo (ref), doenca, formulario
- ■ Questionario — paciente (ref), data, humor (1-5), sintomas, notas
- ■ Consulta — paciente, psicologo, data, estado (agendada/realizada/cancelada)
- ■ Desafio — paciente, psicologo, titulo, descricao, tipo, datas, estado, sugestao
- ■ Mensagem — paciente, psicologo, remetente, mensagem, data
- ■ Forum — titulo, descricao, autor, tags
- ■ MensagemForum — forum (ref), autor, conteudo
- ■ Alerta — paciente, psicologo, tipo, mensagem, nivel, lido

Auth & Security

- ■ verifyToken middleware reads Bearer token and attaches req.user
- ■ verifyTokenByRole restricts routes by tipo
- ■ Passwords hashed with bcrypt (cost factor >= 10) — never stored plain text
- ■ Login compares with bcrypt.compare
- ■ JWT includes id and tipo, signed with JWT_SECRET from .env

Routes & CRUD

- ■ All 11 entities have complete CRUD routes (POST, GET, GET/:id, PUT, DELETE)
- ■ Filtered routes implemented (e.g., /paciente/:id, /psicologo/:id)
- ■ PATCH routes for estado updates on Desafio and Alerta
- ■ Role restrictions applied on every non-public route

Analysis & XML (INFME)

- ■ analiseHumor.ts implements 3 rules (urgente, humor_baixo, sintomas)
- ■ Alertas created automatically in database
- ■ POST /api/analise/paciente/:id/run endpoint works
- ■ historicoPaciente.xsd created in xml/schemas/

- ■ xmlExport.ts generates valid XML and saves to xml/exports/
- ■ xmlValidation.ts validates XML against XSD before returning
- ■ GET /api/export/paciente/:id/historico/xml returns application/xml

Testing

- ■ All 18 Bruno steps pass with correct status codes
- ■ Protected routes return 401 without token
- ■ Forbidden routes return 403 for wrong role
- ■ Duplicate email returns 409
- ■ Invalid IDs return 400
- ■ XML export returns valid, XSD-conformant document

When all boxes above are checked, your MindLink backend is complete and ready for frontend integration.